

PRÉSENTATION

SCRUM en théâtre

Trois piliers : Transparence, Inspection, Adaptation

Auteur : Philippe Laurens

Date : 30 septembre 2026

Version : 1.0

Sommaire

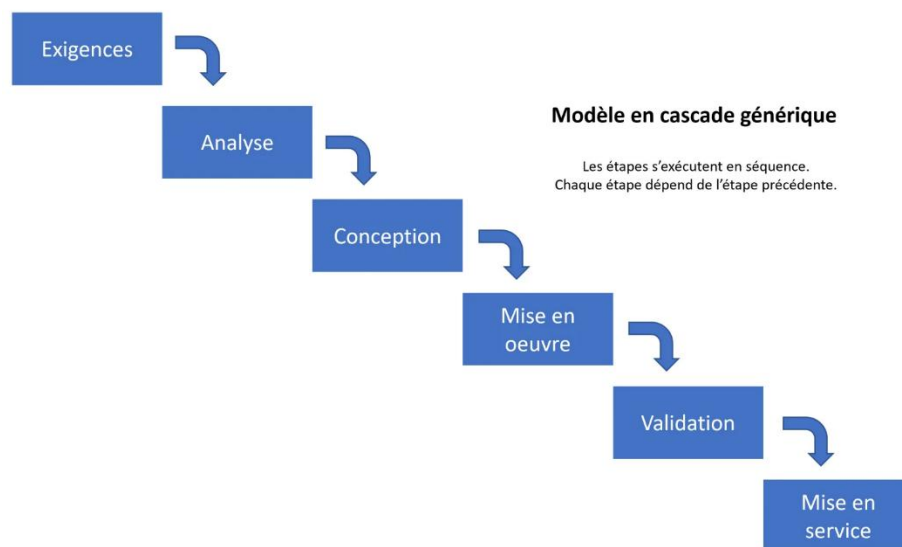
Introduction	3
A. Les cycles de développement	3
B. Le Manifeste Agile.....	4
Partie 1 – La méthode SCRUM	6
1.1 Présentation.....	6
1.2 Les piliers.....	6
1.3 L'équipe Scrum.....	7
Partie 2 – Pratiques et outils	10
2.1 Les événements Scrum	10
2.2 Artefacts de la méthode Scrum	13
2.3 Outils et Éléments de Suivi Complémentaires.....	15
2.4 La méthode Kanban	18
Conclusion.....	19

Introduction

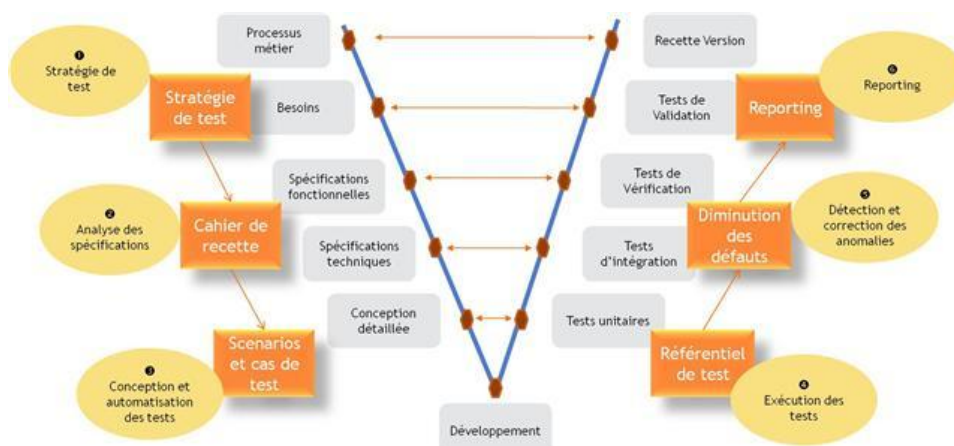
A. Les cycles de développement

Il existe différents types de cycles de développement entrant dans la réalisation d'un logiciel. Ces cycles prendront en compte toutes les étapes de la conception d'un logiciel.

- **Le Modèle en cascade** – Hérité de l'industrie du BTP, il repose sur les hypothèses suivantes : on ne peut pas construire la toiture avant les fondations ; les conséquences d'une modification en amont du cycle ont un impact majeur sur les coûts en aval. Il exécute des phases qui ont pour caractéristiques : de produire des livrables définis au préalable, à une date précise, de ne se terminer que lorsque les livrables sont jugés satisfaisants lors d'une étape de validation.



- **Le Cycle en V.** Le modèle du cycle en V, amélioration du modèle en cascade, permet en cas d'anomalie de limiter un retour aux étapes précédentes. Les phases de la partie montante renvoient de l'information sur les phases en vis-à-vis lorsque des défauts sont détectés afin d'améliorer le logiciel. Il met en évidence la nécessité d'anticiper et de préparer dans les étapes descendantes les « attendus » des futures étapes montantes : ceux des tests de validation sont définis lors des spécifications, ceux des tests unitaires lors de la conception.



- **Le Cycle semi-itératif** (agile). Le cycle semi-itératif a pour origine les travaux de James Martin publiés à partir de 1989 dans diverses revues, et est totalement formalisé en 1991 dans le livre RAD, Développement rapide d'applications. C'est vers 2001 avec l'apparition de plusieurs méthodes dont ASD, FDD, Crystal, Scrum ou l'Extreme Programming et la vision uniformisée de leurs auteurs dans le cadre du Manifeste Agile et de l'Agile Alliance, qu'il est généralisé.

À retenir

En 2001, aux États-Unis, dix-sept figures éminentes du développement logiciel (parmi lesquelles Ken Schwaber, fondateur de Scrum et Jeff Sutherland, qui en fit un succès commercial) se réunissent pour débattre d'un thème unificateur de leurs méthodes respectives. Ils extraient de leurs usages respectifs les critères communs et les principes qui, selon eux, conduisent aux meilleurs concepts de direction de projets et de développement de logiciels. De cette réunion émerge le Manifeste agile, considéré comme la définition canonique du développement agile et de ses principes sous-jacents.

B. Le Manifeste Agile

Les valeurs – Nous découvrons comment mieux développer des logiciels par la pratique et en aidant les autres à le faire. Ces expériences nous ont amenés à valoriser quatre valeurs fondamentales :

- Les individus et leurs interactions, plus que les processus et les outils. Dans l'optique agile, l'équipe est bien plus importante que les outils (structurants ou de contrôle) ou les procédures de fonctionnement. Il est préférable d'avoir une équipe soudée et qui communique, composée de développeurs (éventuellement à niveaux variables), plutôt qu'une équipe composée d'experts fonctionnant chacun de manière isolée. La COMMUNICATION est une notion fondamentale.
- Des logiciels opérationnels, plus qu'une documentation exhaustive. Il est vital que l'application fonctionne. Le reste, et notamment la documentation technique, est une aide précieuse mais NON UN BUT EN SOI : une documentation précise est utile comme MOYEN DE COMMUNICATION. La documentation représente une charge de travail importante, mais peut pourtant être néfaste si elle n'est pas à jour. Il est préférable de commenter abondamment le code lui-même, et surtout de TRANSFÉRER LES CONNAISSANCES au sein de l'équipe.
- La collaboration avec les clients, plus que la négociation contractuelle. Le CLIENT doit être IMPLIQUÉ dans le développement, on ne peut se contenter de négocier un contrat au début du projet, puis de négliger les demandes du client. Le client doit COLLABORER AVEC L'ÉQUIPE et fournir un compte rendu continu sur l'adéquation du logiciel avec ses attentes.
- L'adaptation au changement, plus que le suivi d'un plan. La PLANIFICATION INITIALE et la STRUCTURE DU LOGICIEL doivent être FLEXIBLES afin de permettre l'évolution de la demande du client tout au long du projet. Les premières livraisons du logiciel vont souvent provoquer des demandes d'évolution.

À retenir

L'agilité, c'est une culture de communication et d'adaptation, un état d'esprit qui place les personnes, le produit qui fonctionne, le client et l'adaptation au changement avant les processus, la documentation, les contrats et les plans.

Les principes sous-jacents au Manifeste Agile – Ces quatre valeurs se déclinent en 12 principes généraux communs à toutes les méthodes agiles :

- Notre plus haute priorité est de satisfaire le client en livrant rapidement et régulièrement des fonctionnalités à grande valeur ajoutée : [Spring Backlog](#).
- Réalisez les projets avec des personnes motivées. Fournissez-leur l'environnement et le soutien dont ils ont besoin et faites-leur confiance pour atteindre les objectifs fixés : [Rôle du Scrum Master](#)
- Les utilisateurs ou leurs représentants et les développeurs doivent travailler ensemble quotidiennement tout au long du projet : [Stakeholders + Equipe](#)
- Les processus Agiles encouragent un rythme de développement soutenable. Ensemble, les commanditaires, les développeurs et les utilisateurs devraient être capables de maintenir indéfiniment un rythme constant : [Sprint](#)
- Un logiciel opérationnel est la principale mesure d'avancement : [Iteration \(XP\)](#).
- Livrez fréquemment un logiciel opérationnel avec des cycles de quelques semaines à quelques mois et une préférence pour les plus courts : [Iteration + Sprint](#).
- La méthode la plus simple et la plus efficace pour transmettre de l'information à l'équipe de développement et à l'intérieur de celle-ci est le dialogue en face à face : [Daily Scrum, Sprint Planning Meeting + Sprint Review Meeting + Sprint Retrospective Meeting](#).
- À intervalles réguliers, l'équipe réfléchit aux moyens de devenir plus efficace, puis règle et modifie son comportement en conséquence : [Daily Scrum, Sprint Planning Meeting + Sprint Review Meeting + Sprint Retrospective Meeting](#).
- Accueillez positivement les changements de besoins, même tard dans le projet. Les processus Agiles exploitent le changement pour donner un avantage compétitif au client : BURN-DOWN CHART.
- Une attention continue à l'excellence technique et une bonne conception renforce l'agilité : [l'amélioration Continue](#).
- La simplicité, c'est-à-dire l'art de minimiser la quantité de travail inutile, est essentielle.
- Les meilleures architectures, spécifications, conceptions émergent d'équipes autoorganisées.

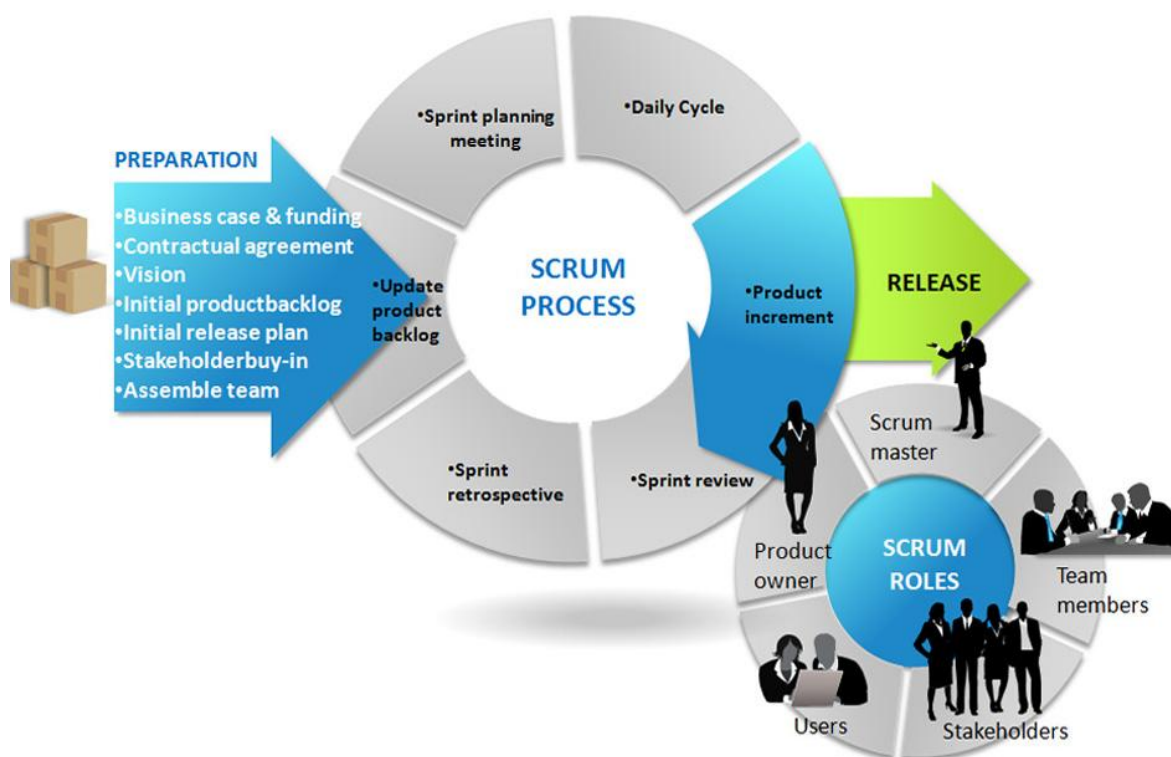
À retenir

Les 12 principes agiles déclinent les quatre valeurs du Manifeste en pratiques concrètes : livrer souvent et en continu un logiciel fonctionnel, travailler en étroite collaboration avec le client, faire confiance à des équipes autoorganisées et motivées, accueillir le changement, et s'améliorer régulièrement dans la simplicité et l'excellence technique.

Partie 1 – La méthode SCRUM

1.1 Présentation

Scrum est une méthodologie dédiée à la gestion de projets, qui permet aux équipes agiles de planifier leurs listes de tâches par blocs de temps appelés sprints, un court bloc de temps (généralement d'une à quatre semaines) que l'équipe peut estimer en termes de valeur apportée au projet et à l'utilisateur. Chaque sprint est guidé par un objectif (le Sprint Goal) : une cible claire, définie en début de sprint, qui donne du sens au travail de l'équipe et précise ce qu'elle s'engage à accomplir. Les tâches retenues dans le sprint backlog sont choisies pour atteindre cet objectif, et l'incrément livré en fin de sprint permet de vérifier s'il a été atteint.



1.2 Les piliers

En plus des valeurs et principes communs à toutes les méthodes agiles, issus du Manifeste Agile, la méthode Scrum s'appuie sur trois piliers fondamentaux : la transparence, l'inspection et l'adaptation.

- **La transparence** – Scrum met l'accent sur le fait d'avoir un langage commun entre l'équipe et le management. Ce langage commun doit permettre à tout observateur d'obtenir rapidement une bonne compréhension du projet.
- **L'inspection** – À intervalle régulier, Scrum propose de faire le point sur les différents artéfacts produits, afin de détecter toute variation indésirable. Ces inspections ne doivent pas être faites trop fréquemment, ou par un inspecteur mal formé : cela nuirait à l'avancement du projet.
- **L'adaptation** – Si une dérive est constatée pendant l'inspection, le processus doit être adapté. Scrum fournit des rituels, durant lesquels cette adaptation est possible : les réunions de planification de sprint, les daily scrums, la revue de sprint ainsi que de la rétrospective de sprint.

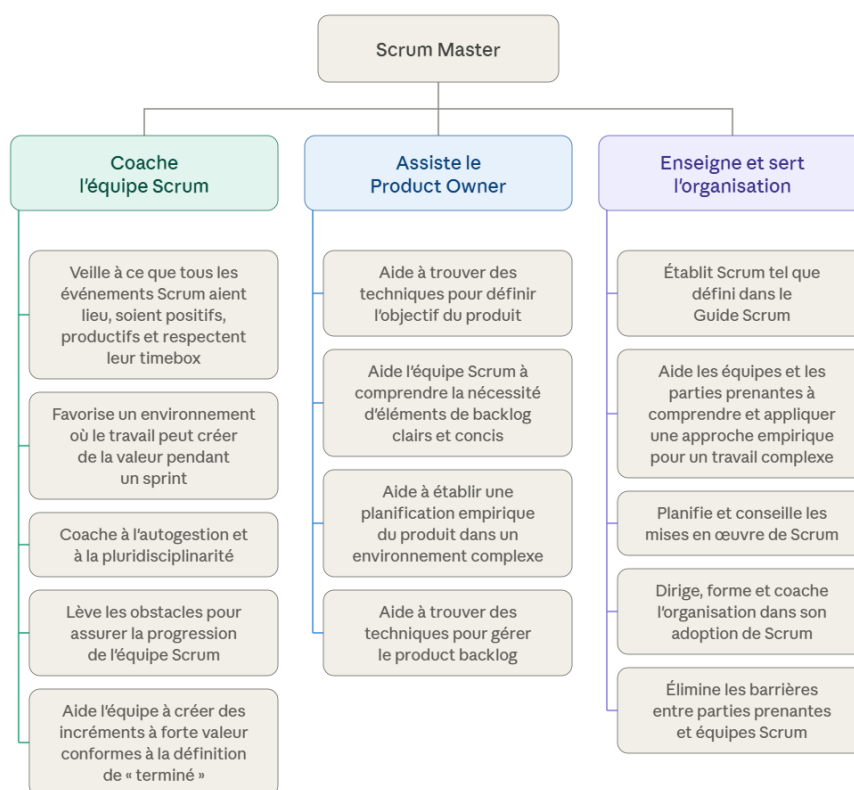
1.3 L'équipe Scrum

Une équipe Scrum est composée de trois rôles majeurs : le Scrum Master (s'assurant que l'équipe est fonctionnelle et productive), le Product Owner (responsable de la vision produit et priorisant les besoins), l'équipe de réalisation (auto-organisée et responsable, chargée de développer l'application).

1.3.1 Le Scrum Master

Le Scrum Master possède le rôle de facilitateur.

- **Responsabilité à l'égard des processus** – Il est le garant de l'application de la méthode Scrum. Il s'assure que les règles sont adoptées, comprises et appliquées par tous. Il a toute autorité sur les décisions à prendre concernant l'application de la méthode Scrum.



- **Collaboration avec le Product Owner** – L'une des tâches incombant au Scrum Master est l'étroite collaboration avec le Product Owner dans la rédaction du Product Backlog. A ce titre, il se doit de l'aider dans la phase d'écriture des User Stories et peut également le conseiller sur la priorité à donner sur chacune d'entre elles : mais il n'a qu'un devoir de conseil.

Servir l'équipe – Le rôle le plus important est d'assurer une parfaite coopération entre tous les membres du projet. Cette coopération est la clé de la réussite d'un projet utilisant la méthode Scrum. L'objectif de Scrum est de rendre l'équipe autonome et responsable, il faut donc veiller à ce qu'elle travaille dans un environnement de bonne qualité et supprimer tous les irritants.

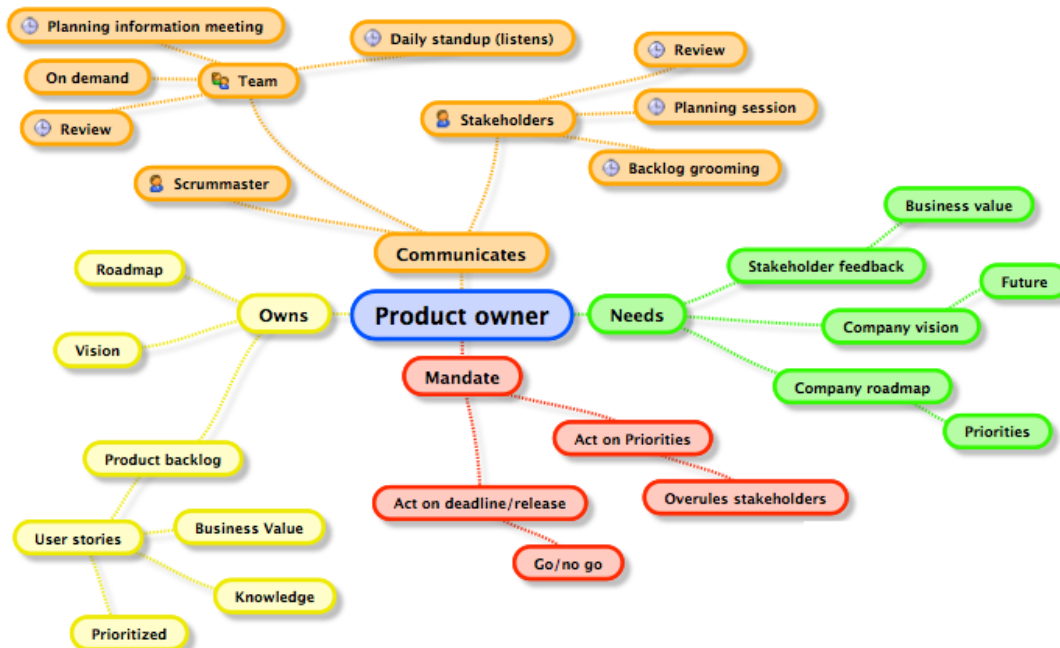
À noter

En complément de ses responsabilités, le Scrum Master se doit d'animer les réunions composant le cérémonial Scrum : Planification de Sprint (phase de préparation), le Scrum Meeting (réunion quotidienne), la revue de Sprint (en fin de Sprint), la rétrospective de Sprint (en fin de Sprint).

1.3.2 Le Product Owner

Le Product Owner est la personne représentant la vision du produit vis-à-vis du Scrum Master et de l'équipe : il centralise à lui seul l'ensemble des besoins des utilisateurs afin d'en être l'unique représentant.

- **Définir la vision du produit** – Il se doit de définir la vision du produit répondant à un ou plusieurs problèmes et pilotée par objectifs. Cette vision se doit d'être partagée et portée par les utilisateurs, leur permettant ainsi de se projeter dans le futur. Le Product Owner doit être la voix unique de ces utilisateurs vis-à-vis du Scrum Master et de l'équipe.



- **Responsable du backlog** – Le Product Owner est le seul responsable du Product Backlog, il porte la responsabilité de l'écriture des User Stories et de leur priorisation. Le Scrum Master peut aider à cette tâche via l'animation d'atelier, mais seul le Product Owner portera la responsabilité de prioriser l'une ou l'autre fonctionnalité parmi l'ensemble du Product Backlog.
- **Implication dans le processus Scrum** – Le Scrum Master et l'équipe ne peuvent s'engager sur une date de livraison et sur le périmètre d'une Release. C'est au Product Owner de définir le périmètre visé par la Release et la date de livraison de celle-ci. Le Product Owner doit s'impliquer dans l'ensemble du processus Scrum en participant à l'ensemble des cérémoniaux, mais également en collaborant avec l'équipe. Il doit être disponible à chaque instant du projet.

Le Product Owner a pour pouvoir de choisir le contenu du Product Backlog et de définir le contenu des Sprints et Release, modifiant ainsi les priorités et fonctionnalités de chaque Sprint. Mais lorsqu'un Sprint est commencé, la modification et fonctionnalités n'est plus possible. Lors de la revue de Sprint, il a la possibilité d'accepter ou non la livraison effectuée par l'équipe de développement, en se basant sur des indicateurs qui lui sont propres mais partagés avec tous les membres de l'équipe Scrum.

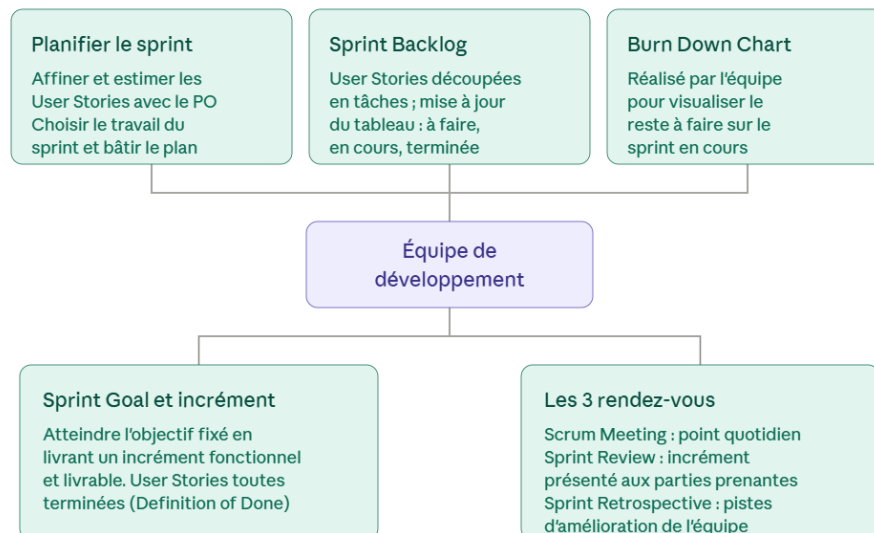
À retenir

Le Product Owner a pour pouvoir de choisir le contenu du Product Backlog et de définir le contenu des Sprints et Release. Il peut ainsi modifier les priorités et fonctionnalités de chaque Sprint

1.3.3 L'équipe

L'équipe a pour fonction principale la réalisation des User Stories contenues dans le Sprint Backlog.

- **Participer à la planification du sprint** – L'équipe : elle aide le Product Owner à préciser et à estimer les User Stories avant qu'elles entrent dans un sprint. Elle choisit ce qu'elle peut réaliser et construit son plan pour atteindre le Sprint Goal. Le Sprint Backlog en est le résultat.



- **Le Sprint Goal et l'incrément** – La première responsabilité de l'équipe de développement est d'atteindre l'objectif fixé lors de la planification du sprint (le Sprint Goal) en livrant, à la fin de chaque sprint, un incrément fonctionnel et potentiellement livrable. Cet objectif est pleinement atteint lorsque l'ensemble des User Stories prévues dans le sprint ont été réalisées et jugées comme terminées, conformément au Definition of Done)
- **Participation au Scrum Meeting, à la Sprint Review et à la Sprint Retrospective** – Le Scrum Meeting quotidien permet à chacun d'exposer son avancement, ses difficultés et ses projets du jour. Le Sprint Review présente l'incrément aux parties prenantes pour recueillir leurs retours. La Sprint Retrospective identifie les pistes d'amélioration de l'équipe. Ils sont essentiels à la transparence et à l'adaptation et ne doivent pas être négligés.
- **Le Sprint Backlog** – Le Sprint Backlog est composé d'User Stories elles-mêmes découpées en tâches. Il incombe à l'équipe de développement de faire vivre le Sprint Backlog en indiquant pour chaque tâche son état d'avancement (à faire, en cours, terminée) via la mise à jour du tableau représentatif du Sprint Backlog.
- **Burn Down Chart** – Il incombe aux membres de l'équipe de réaliser le Burn Down Chart permettant de visualiser le reste à faire sur le Sprint en cours.

L'auto-organisation a pour but de renforcer le degré de responsabilité des équipes au quotidien. Ainsi, chaque membre de l'équipe a la capacité de choisir la tâche qu'il va développer, d'en choisir l'aspect technique et bien d'autres éléments.

À noter

L'équipe ne devient pas autonome et responsable immédiatement, le Scrum Master doit donc la guider en veillant notamment à la santé de celle-ci, sans oublier la mise en place de l'amélioration continue.

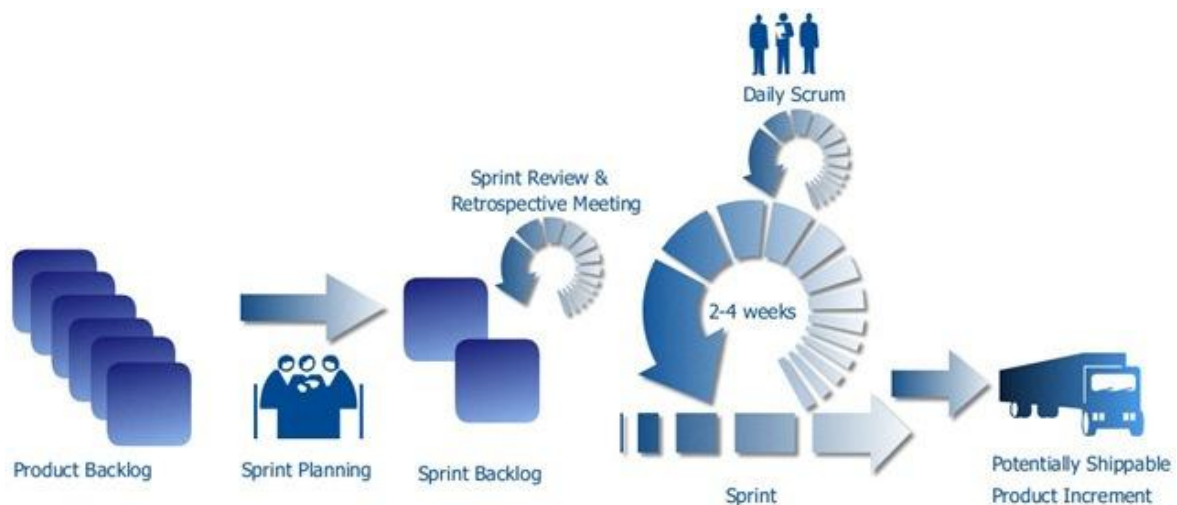
Partie 2 – Pratiques et outils

2.1 Les événements Scrum

La méthode Scrum s'appuie sur cinq événements clés qui rythment le travail de l'équipe, favorisent l'inspection continue et garantissent la transparence à chaque étape du projet.

2.1.1 Sprint

Le Sprint est le cœur de la méthode Scrum. C'est un événement conteneur d'une durée fixe et courte (généralement d'une à quatre semaines) durant lequel l'équipe réalise un ensemble de fonctionnalités prêtes à être livrées.



- Ses caractéristiques clés sont :
 - **Durée fixe (Timebox) :** Sa longueur ne change jamais en cours de route. Dès qu'un Sprint se termine, le suivant commence immédiatement.
 - **Sprint Goal :** L'équipe s'engage sur un objectif précis à atteindre à la fin de la période.
 - **Stabilité :** Aucun changement susceptible de remettre en cause l'objectif initial n'est accepté durant sa réalisation.
- Chaque Sprint contient l'ensemble des autres événements Scrum (planification, mêlée quotidienne, revue et rétrospective).

2.1.2 Sprint Planning et Sprint Backlog

Dans Scrum, chaque itération commence par la réunion de planification du Sprint (Sprint Planning). Lors de cette réunion, le Product Owner et l'équipe négocient les user stories que l'équipe traitera pendant le Sprint. Limité à un temps fixe de quatre heures, cet échange prend la forme d'une discussion entre le Product Owner et l'équipe : le Product Owner décide quelles stories présentent la priorité la plus élevée pour la livraison et généreront la plus forte valeur métier. L'équipe a le pouvoir de faire des remarques, d'exprimer des réserves ou de signaler des obstacles.

Cette réunion collaborative au cours de laquelle l'ensemble de l'équipe Scrum définit la valeur à produire permet de sélectionner les éléments prioritaires du Product Backlog et établit le plan d'action nécessaire pour les réaliser, aboutissant ainsi à la création formelle du Sprint Backlog.

- **Scrum Task Board** – On peut rendre le Sprint Backlog visible en l'affichant sur un tableau de tâches Scrum : les membres de l'équipe mettent à jour le tableau en continu tout au long du Sprint ; si quelqu'un pense à une nouvelle tâche, il rédige une nouvelle carte et la colle sur le mur. Que ce soit pendant ou avant la mêlée quotidienne (Daily Scrum), les estimations sont révisées (à la hausse ou à la baisse) et les cartes sont déplacées sur le tableau.

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... DC 8
	Code the... 2	Code the... 8	Test the... SC 8		Test the... SC 8
	Test the... 8	Test the... 4			Test the... SC 6
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 8
	Code the... 4	Code the... 6			Test the... SC 6

Chaque carte de tâche commence sur le tableau de tâches Scrum dans la colonne « À faire » (To Do). Les colonnes que nous utilisons généralement sur un tableau de tâches sont :

- User Story : la description de la story (« En tant qu'utilisateur, je souhaite... »)
 - To Do : les cartes qui ne se trouvent ni dans la colonne Done, ni dans la colonne In Process (ou To Verify) pour le Sprint actuel.
 - Work In Process : toutes les cartes en cours de traitement vont ici. Le développeur qui choisit de travailler sur une tâche la déplace dans cette colonne lorsqu'il est prêt à la démarrer. Cela se produit souvent pendant le Daily Scrum, lorsque quelqu'un dit : Aujourd'hui, je vais travailler sur cette fonctionnalité.
 - To Verify : de nombreuses tâches ont des cartes de tests correspondantes.
 - Done : les cartes s'accumulent ici lorsqu'elles sont achevées. Elles sont retirées à la fin du Sprint. Il arrive parfois que l'on en retire une partie ou la totalité en cours de Sprint si le nombre de cartes devient trop important.
- **Definition of Done** – Lorsqu'un élément du *Product Backlog* ou un incrément est décrit comme Done, chacun doit comprendre ce que Done signifie. Bien que cela varie considérablement d'une équipe Scrum à l'autre, les membres doivent avoir une compréhension commune de ce que signifie l'achèvement du travail afin de garantir la transparence.
- Il s'agit de la Definition of Done de l'équipe Scrum : elle sert à évaluer quand le travail sur l'incrément de produit est terminé. Cette même définition guide l'équipe pour savoir combien d'éléments du Product Backlog elle peut sélectionner lors du Sprint Planning. L'objectif de chaque Sprint est de livrer des incréments de fonctionnalités potentiellement livrables qui respectent la Definition of Done actuelle.

2.1.3 Daily Scrum

Le Daily Scrum (La mêlée quotidienne) est un point de synchronisation rapide de 15 minutes au cours duquel l'équipe inspecte l'avancement vers l'objectif de Sprint et ajuste son plan pour la journée. Conçu pour favoriser la transparence et la réactivité, le Daily permet à chaque membre de partager ses réussites de la veille, son plan du jour et les éventuels obstacles rencontrés.

Trois questions-type :

- Qu'ai-je accompli hier qui a aidé l'équipe à atteindre l'objectif du Sprint ?
- Que vais-je faire aujourd'hui pour aider l'équipe à atteindre l'objectif du Sprint ?
- Est-ce que je vois des obstacles qui m'empêchent, moi ou l'équipe, d'atteindre cet objectif ?

2.1.4 Scrum Review

Chaque Sprint doit obligatoirement livrer un incrément de produit potentiellement déployable. Cela signifie qu'à la fin de chaque Sprint, l'équipe a produit un élément logiciel codé, testé et utilisable. Une réunion de revue de Sprint est donc organisée à l'issue de chaque itération. Au cours de celle-ci, l'équipe Scrum présente ce qu'elle a accompli. En général, cela prend la forme d'une démonstration des nouvelles fonctionnalités.

La réunion de revue de Sprint est volontairement très informelle. On y interdit souvent l'utilisation de présentations PowerPoint et l'on limite le temps de préparation à deux heures maximum. Elle ne doit pas devenir une distraction ni constituer une charge de travail importante pour l'équipe, mais découler naturellement du Sprint.

Les participants à cette revue incluent généralement le Product Owner, l'équipe Scrum, le Scrum Master, la direction, les clients et des développeurs d'autres projets. Durant la réunion, le projet est évalué au regard de l'objectif de Sprint déterminé lors du Sprint Planning. Idéalement, l'équipe a terminé chaque élément du Product Backlog sélectionné pour le Sprint, mais le plus important reste l'atteinte de l'objectif global du Sprint.

2.1.5 Scrum Retrospective

Chaque Sprint se termine par une rétrospective. Lors de cette réunion, l'équipe mène une réflexion sur ses propres processus. Elle analyse son fonctionnement et définit des actions pour l'adapter lors des futurs Sprints.

Une rétrospective approfondie nécessite un climat de sécurité psychologique absurde dans la plupart des organisations. Sans cette sécurité, la discussion contournera les sujets délicats ou dérivera vers la recherche de coupables et l'hostilité.

Les rétrospectives mettent souvent en lumière des blocages organisationnels. Une fois que l'équipe a résolu les problèmes sous son influence directe, le Scrum Master doit s'efforcer d'élargir ce champ d'action pour réduire progressivement les obstacles à l'échelle de l'entreprise.

Les Scrum Masters peuvent utiliser des techniques variées pour animer les rétrospectives, comme l'écriture en silence, les frises chronologiques (timelines) ou les histogrammes de satisfaction. Dans tous les cas, les objectifs sont de parvenir à une compréhension commune des différents points de vue et d'élaborer des actions qui permettront à l'équipe de passer au niveau supérieur.

2.2 Artefacts de la méthode Scrum

Les artefacts officiels représentent de la valeur ou du travail sous forme d'éléments ayant subi des transformations progressives tout au long du cycle de développement.

2.2.1 Product Backlog

Le Product Backlog s'apparente davantage à une structure de découpage du produit (Product Breakdown Structure) qu'à une structure de découpage du travail (Work Breakdown Structure). Se concentrer sur les capacités du produit plutôt que sur les tâches de travail permet de conserver un backlog orienté vers les objectifs, mettant l'accent sur le quoi plutôt que sur le comment.

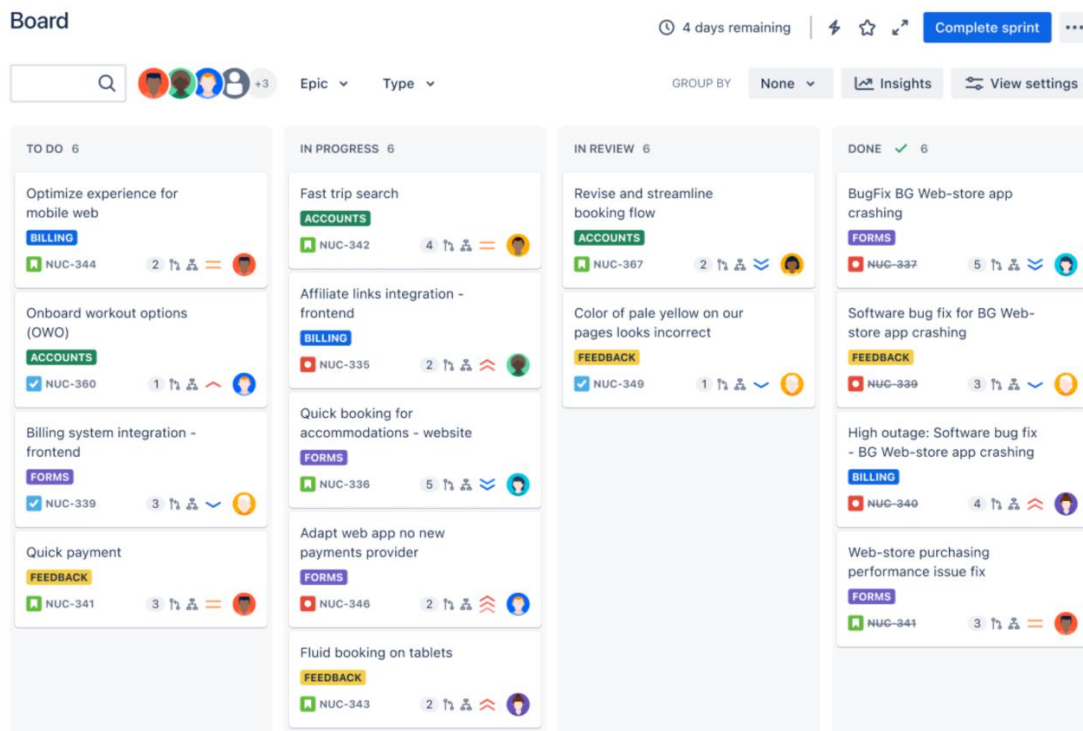
- **Product Backlog Item** – Il est très courant de trouver des éléments sous la forme de :



- User stories : représentent de nouvelles fonctionnalités.
 - Bugs (ou Anomalies) : représentent le travail nécessaire pour corriger un défaut.
 - Chores (Tâches techniques / corvées) : représentent du travail indispensable à réaliser, mais qui n'apporte pas de valeur métier directe.
 - Epics : représentent de grands récits utilisateurs. Définis à l'origine comme étant trop volumineux pour tenir dans une seule itération.
 - Prototypes : représentent des proofs of concept permettant de collecter des informations afin de décider si une fonctionnalité présente de la valeur.
- **Product Backlog Refinement** – L'affinage du Product Backlog est l'action d'ajouter des détails, des estimations et de l'ordre aux éléments du Product Backlog. Il s'agit d'un processus continu au cours duquel le Product Owner et l'équipe de développement collaborent sur les détails des éléments du Product Backlog.
 - Durant l'affinage du Product Backlog, les éléments sont examinés et révisés. L'équipe Scrum décide de la manière et du moment où cet affinage est réalisé. L'affinage ne consomme généralement pas plus de 10 % de la capacité de l'équipe de développement. Cependant, les éléments du Product Backlog peuvent être mis à jour à tout moment par le Product Owner ou à sa discrétion.
 - Les éléments situés en haut du Product Backlog (les plus prioritaires) sont généralement plus clairs et plus détaillés que ceux situés plus bas. Des estimations plus précises sont établies grâce à cette clarté accrue et ces détails supplémentaires : les éléments du Product Backlog qui occuperont l'équipe de développement pour le Sprint à venir sont affinés de sorte que chaque élément puisse raisonnablement être Terminé (Done) dans le respect du temps imparti (time-box) du Sprint. Les éléments du Product Backlog qui peuvent être « Terminés » par l'équipe de développement au cours d'un seul Sprint sont considérés comme Prêts (Ready) à être sélectionnés lors de la planification du Sprint (Sprint Planning).

2.2.2 Sprint Backlog

Le Sprint Backlog est un plan d'action dynamique composé du Sprint Goal, de l'ensemble des éléments du Product Backlog sélectionnés pour l'itération, des tâches techniques nécessaires pour les réaliser. Créé lors de la réunion de Sprint Planning, il appartient exclusivement à l'équipe de développement qui le met à jour quotidiennement pour refléter la progression du travail vers l'objectif fixé.



- **Il apporte de la transparence** – En rendant le travail et la charge restant à accomplir entièrement visibles sur le Scrum Board, il garantit que toute l'équipe et les parties prenantes partagent la même vision de l'état d'avancement du Sprint en temps réel.
- **Il matérialise une transformation** – En tant qu'artefact, il matérialise le passage des besoins abstraits issus du Product Backlog vers des tâches concrètes qui se traduiront à la fin du Sprint par un incrément de produit utilisable et terminé.
- **Il repose sur un engagement fort** : Chaque artefact Scrum contient un engagement explicite garantissant sa qualité et son orientation ; pour le Sprint Backlog, cet engagement est le Sprint Goal, qui donne du sens et un cap clair au travail de l'équipe tout au long de l'itération.

2.2.3 L'Incrément

L'Incrément est la somme de tous les éléments du Product Backlog complétés au cours d'un Sprint, combinée à la valeur de l'ensemble des Sprints précédents. Il s'agit d'une version concrète, utilisable et entièrement testée du produit qui apporte une valeur réelle aux utilisateurs. À la fin de chaque Sprint, le nouvel incrément doit être immédiatement déployable en production, que le Product Owner décide ou non de le publier.

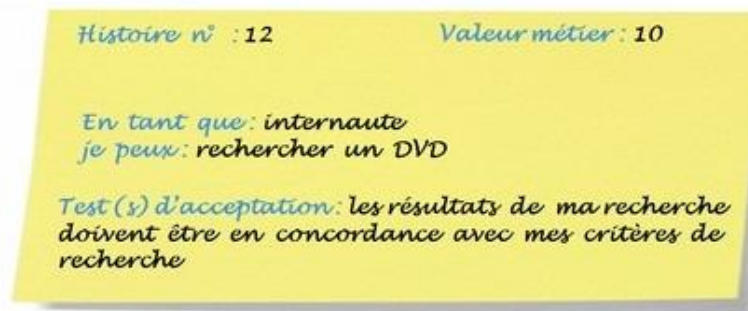
Il représente l'aboutissement du cycle Scrum où des besoins abstraits (du Product Backlog) et du travail planifié (du Sprint Backlog) se transforment en une fonctionnalité concrète, intégrée et fonctionnelle.

2.3 Outils et Éléments de Suivi Complémentaires

La réussite agile repose sur la maîtrise des outils de pilotage au quotidien : de la formulation des besoins à travers les User Stories, au suivi quotidien de l'avancement via le Burndown Chart, et le contrat d'exigence qualité via la DoD (Definition of Done).

2.3.1 Les User Stories

Une User Story est une fonctionnalité élémentaire décrite du point de vue de l'utilisateur. Elle apporte de la valeur métier et peut être réalisée en moins d'une itération de deux semaines : elles sont généralement issues de la décomposition de fonctionnalités plus larges. Leur écriture s'appuie sur le formalisme suivant : « En tant que [rôle], je veux [action], afin de [bénéfice]. »



- Certains attributs y sont récurrents :
 - Le rôle (qui)
 - L'action ou le besoin fonctionnel (quoi)
 - La justification (pourquoi), parfois facultative si le bénéfice est évident
 - La business value, qui représente le bénéfice apporté à l'utilisateur ou à l'organisation
 - L'estimation en Story Points, qui évalue l'effort et la complexité nécessaires à sa réalisation.

On y associe également des tests d'acceptation (conditions de satisfaction). En fin d'itération, l'User Story est considérée comme terminée uniquement si l'ensemble de ces tests est validé.

Le modèle classique des 3C (Card, Conversation, Confirmation) est la base théorique qui explique pourquoi une User Story n'est pas un cahier des charges rigide, mais un support de discussion.

- Carte (Card) – Les User Stories sont rédigées sur des cartes. La carte ne contient pas l'ensemble des informations détaillant le besoin, mais juste assez de texte pour identifier l'exigence et rappeler à tous l'objet de la story. La carte est un jeton représentant le besoin, utilisé lors de la planification. Des annotations y sont portées pour refléter sa priorité et son coût.
- Conversation – Le besoin lui-même est transmis des clients (ou du Product Owner) aux développeurs par le dialogue : un échange d'idées, d'opinions et de points de vue. Cette conversation s'étale dans le temps, notamment lors de l'estimation de la story, puis lors de la réunion de planification d'itération lorsque sa réalisation est programmée
- Confirmation – Quels que soient le niveau de discussion ou le volume de documentation, la certitude absolue sur ce qui doit être réalisé reste difficile à atteindre. La confirmation apporte la garantie nécessaire : il s'agit des tests d'acceptation. Au début de l'itération, le client explique aux développeurs comment il vérifiera que le travail répond au besoin, en définissant les tests d'acceptation qui serviront à valider la bonne mise en œuvre de la story.

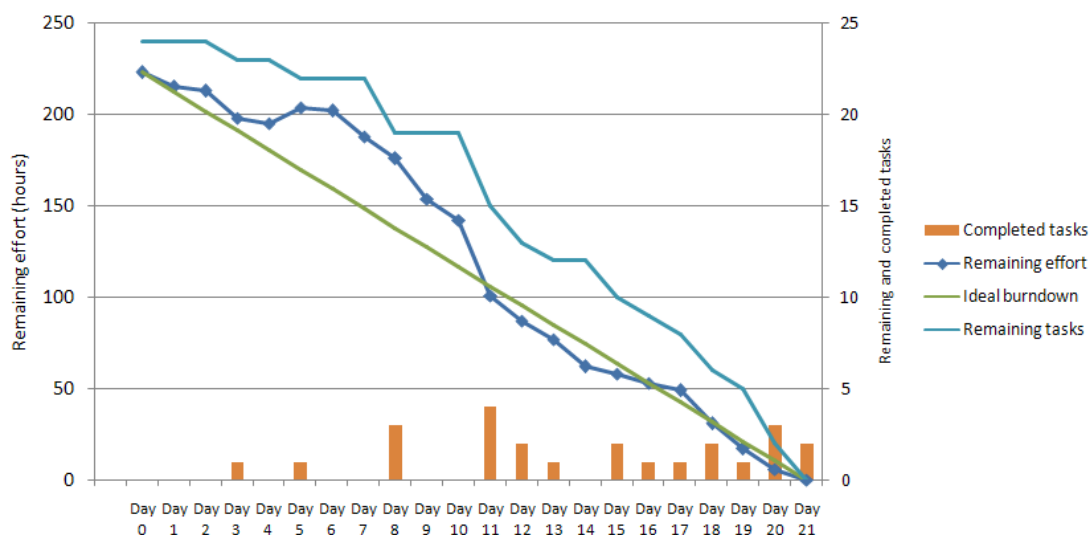
2.3.2 Story point and Planning poker

Un Story Point est une unité de mesure relative utilisée par les équipes agiles pour estimer l'effort global nécessaire à la réalisation d'une User Story. Contrairement aux estimations traditionnelles exprimées en heures ou en jours, les Story Points n'évaluent pas le temps passé, mais combinent trois facteurs : la complexité de la tâche à accomplir, la quantité de travail à réaliser, et l'incertitude et les risques techniques associés.

- L'estimation relative – L'équipe ne cherche pas à savoir combien d'heures prendra une tâche, mais la compare à une autre tâche de référence déjà connue : si créer un formulaire simple vaut 2 points, créer une page de paiement complète, deux fois plus complexe, vaudra 5 ou 8 points.
- La suite de Fibonacci – Les points sont généralement attribués en suivant une échelle spécifique (souvent inspirée de la suite de Fibonacci) : 1, 2, 3, 5, 8, 13, 21... Plus l'élément est gros, plus l'incertitude augmente, ce qui force à espacer les valeurs.
- Le Planning Poker – C'est une technique ludique et collaborative la plus répandue pour estimer les Story Points au sein d'une équipe agile. Dérivée de la méthode Delphi, elle s'appuie sur le consensus pour éviter les biais d'influence (comme l'avis du développeur le plus senior ou le plus véhément).

2.3.3 Le Burn-Down chart

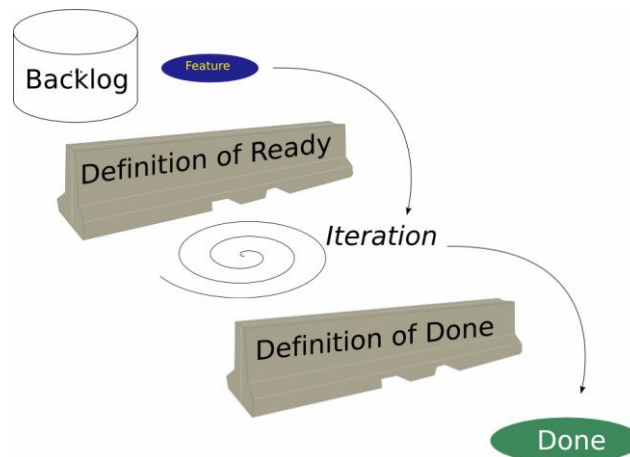
Différentes pratiques de projection fondées sur les tendances sont utilisées pour anticiper le progrès, telles que les burn-down charts, les burn-up charts ou les diagrammes de flux cumulé (cumulative flow diagrams).



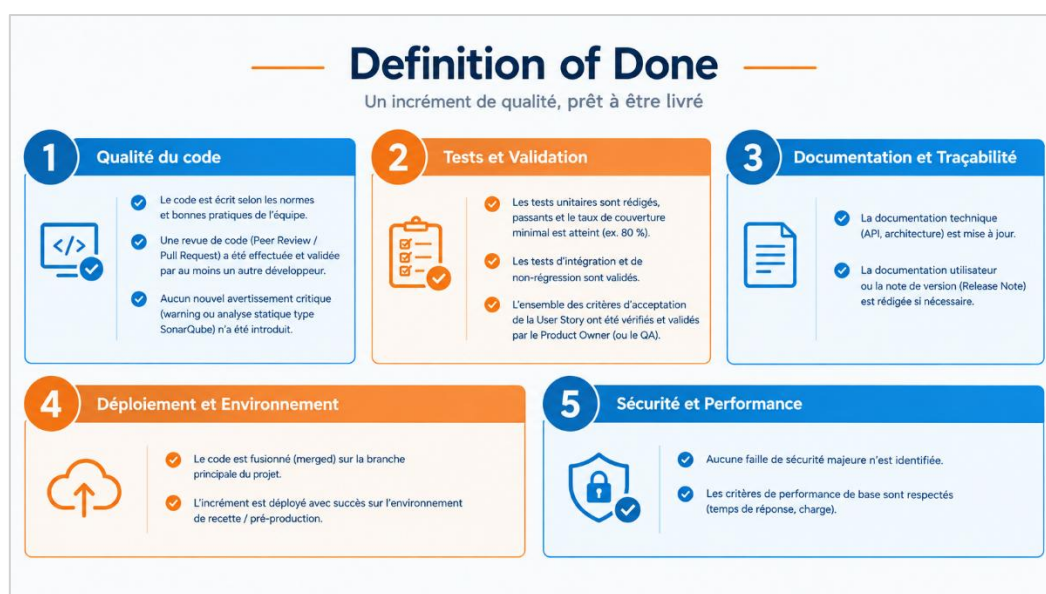
- Le Product Owner suit la quantité totale de travail restant au moins lors de chaque revue de Sprint (Sprint Review). Il compare ce volume au travail restant lors des revues précédentes afin d'évaluer le progrès vers l'achèvement du travail prévu dans les délais visés pour l'objectif. Cette information est rendue transparente pour l'ensemble des parties prenantes.
- L'équipe de développement suit ce travail restant au moins lors du Daily Scrum afin d'évaluer la probabilité d'atteindre l'objectif du Sprint Goal. En suivant le travail restant tout au long du Sprint, l'équipe de développement peut piloter son avancement.

2.3.4 Definition of Ready, Definition of Done

La réussite d'un Sprint repose sur des règles du jeu claires et partagées. La Definition of Ready (DoR) et la Definition of Done (DoD) constituent les deux grilles de critères indispensables qui encadrent le travail de l'équipe : la première définit ce qui est « prêt à être démarré » lors du Sprint Planning, la seconde établit ce qui est « réellement terminé » à la fin du Sprint. »

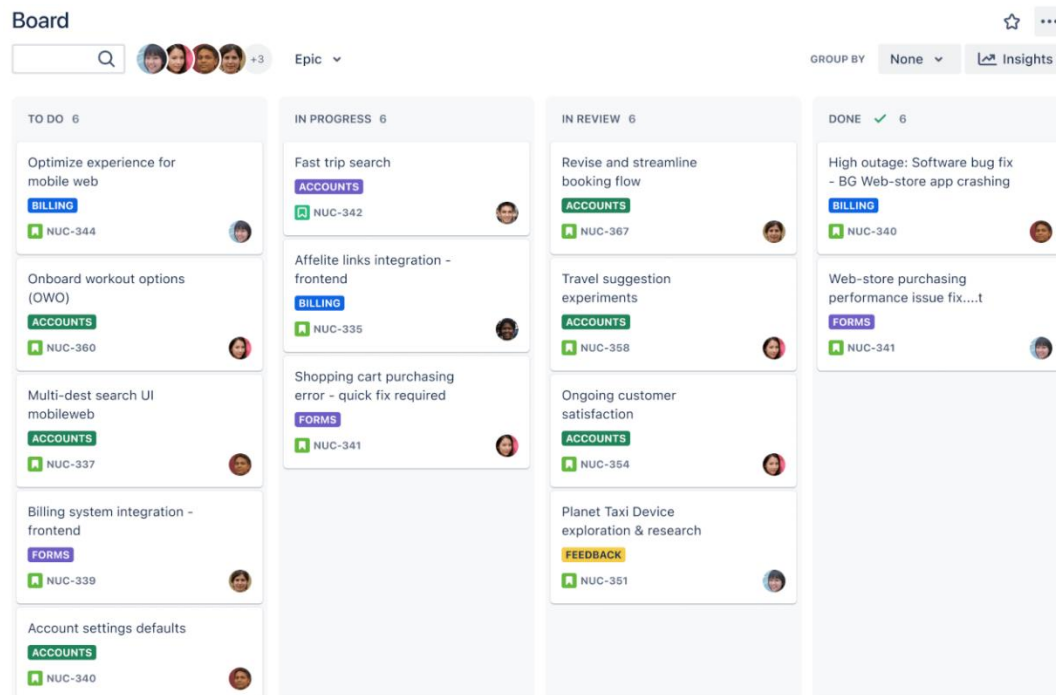


- **Definition of Ready** – Face au risque de blocage lorsque le Product Backlog manque d'éléments exploitables, l'équipe et le Product Owner doivent définir ensemble une Definition of Ready (DoR). Il s'agit d'un ensemble de critères de maturité (description claire, critères d'acceptation, dépendances identifiées, estimation de l'effort, etc.) qu'une User Story doit valider avant d'entrer en Sprint. L'objectif est de trouver un juste équilibre fondé sur le bon sens pour garantir un démarrage efficace et sécuriser la livraison de l'itération.
- **Definition of Done** – La Definition of Done (DoD) est un contrat d'exigence qualité partagé et explicite au sein de l'équipe Scrum. Elle regroupe l'ensemble des critères techniques, fonctionnels et organisationnels indispensables à valider pour qu'un élément du backlog soit considéré comme réellement terminé et intégré à l'incrément de produit utilisable. C'est le garde-fou de sortie, qui garantit l'absence de dette technique et assure que le livrable est prêt à être déployé en production à tout moment, sans travail résiduel.



2.4 La méthode Kanban

La méthode Kanban est un outil de gestion de projet que les équipes peuvent utiliser pour organiser les workflows et les tâches de manière structurée et centralisée. Un tableau Kanban se compose de différentes colonnes représentant les différentes étapes d'un workflow (« To Do », « In Progress », « In Review » et « Done ») et contient des tâches ou tâches individuelles



- Le pilotage par le flux continu (Continuous Flow)** – La différence fondamentale entre Scrum et Kanban réside dans la gestion du temps et la cadence de livraison. Alors que Scrum structure le travail autour de timeboxes rigides (Sprints de 1 à 4 semaines), Kanban repose sur un modèle de flux tendu et ininterrompu : il n'y a pas d'engagement sur une période donnée :
 - Les tâches sont ordonnées par priorité dans le backlog : dès qu'un membre de l'équipe termine une tâche et libère de la capacité dans la colonne In Progress, il tire l'élément le plus prioritaire du backlog vers le tableau. La livraison ne se fait pas en bloc à la fin d'une itération, mais au fil de l'eau, dès qu'une tâche atteint la colonne Done.
 - Le backlog peut être réorganisé à tout moment. Si une urgence absolue survient (bug critique en production, nouvelle réglementation), elle est placée au sommet du backlog et sera prise en charge dès qu'un développeur se libère, sans attendre le prochain Sprint.
- Les métriques d'optimisation du cycle (Lead Time & Cycle Time)** – Plutôt que de mesurer la capacité de production à l'aide de Story Points et de la Vitesse comme en Scrum, la méthode Kanban s'appuie sur la physique des flux. Elle évalue l'efficacité du système en mesurant des durées réelles d'acheminement du travail. Les deux indicateurs fondamentaux sont le Lead Time (temps d'écoulement total) et le Cycle Time (temps de cycle).
 - Le Lead Time mesure le temps total écoulé entre le moment où une demande est exprimée par le client (ou ajoutée au Product Backlog) et le moment où elle est effectivement livrée en production, prête à être utilisée.

- Le Cycle Time mesure le temps nécessaire pour accomplir le travail une fois que l'équipe a commencé à traiter la tâche (passage de la colonne To Do à la colonne In Progress).

Un écart important entre le Lead Time et le Cycle Time indique que les tâches passent l'essentiel de leur vie à attendre sans que personne ne s'en occupe. Au lieu d'estimer chaque tâche individuellement, l'équipe utilise l'historique de son Cycle Time moyen pour s'engager sur des délais fiables auprès du métier

Conclusion

Scrum s'inscrit dans la continuité du Manifeste Agile : il remplace la rigidité des cycles de développement traditionnels par un cadre itératif, fondé sur la transparence, l'inspection et l'adaptation.

Son efficacité repose sur la complémentarité de trois rôles (Scrum Master, Product Owner, équipe), de cinq événements rythmés par le Sprint, et de trois artefacts (Product Backlog, Sprint Backlog, Incrément), soutenus par des outils comme les User Stories, le Planning Poker, le Burn-Down chart et les Definitions of Ready et of Done.

Au-delà des pratiques, la réussite de Scrum tient avant tout à l'adhésion de l'équipe à ses valeurs et à sa capacité d'amélioration continue.

A retenir

Kanban, par son flux continu et sa limitation du travail en cours, offre une alternative ou un complément pertinent selon la nature des projets.